Agentic AI Regulations Corpus and Implementation Evidence

Implementation Specification v1.0

SAP-OSS AI Governance Working Group

April 2026

Document status and purpose

This specification formalises the regulations, frameworks, and conformance tooling relevant to agentic AI deployments in the SAP OSS finance stack. Its three goals are:

1. **Make the regulations machine-readable.** Every source document in docs/regulations/source/ is accompanied by structured JSON records under docs/regulations/structured/ that capture sections, normative requirements, and observational claims.
2. **Bind each requirement to evidence in the repository.** The requirement schema carries an `implementation[]` array of file paths, configuration fields, and documentation anchors, plus a `verification[]` array pointing to AI Verify plugins, Moonshot benchmark IDs, or repository tests. A local validator (docs/regulations/structured/validate.py) fails when any of these paths no longer resolve.
3. **Give reviewers a single printable reference.** This document cross-links the corpus records, the two vendored conformance frameworks, and the code they exercise, so that governance reviews, internal audit, and external assessors can read one PDF instead of navigating three directory trees.

In scope. The Monetary Authority of Singapore (MAS) / AI Verify Foundation *Model AI Governance Framework for Agentic AI* v1.0 (22 Jan 2026); the multi-institution *2025 AI Agent Index*; the Ju & Aral field experiment *Collaborating with AI Agents* (arXiv 2503.18238v3, Feb 2026); and the two vendored open-source conformance tools `aiverify` (v2.0) and `moonshot-cicd` (v1.1.0).

Out of scope. Binding legislation (EU AI Act, US executive orders); sector-specific prudential rules (Basel, Solvency); deterministic non-LLM agent platforms. Those are tracked separately and may be added as additional regulation records without changing the schema.

Audience. AI governance leads, internal audit, model risk, and technical reviewers responsible for evidencing conformance for agentic deployments that use the SAP AI Core / PAL / BTP stack captured in docs/regulations/../tb/ and docs/regulations/../arabic/.

Relationship to other corpora. This corpus is a peer of docs/regulations/../arabic/ (Arabic invoice processing) and docs/regulations/../tb/ (trial balance review). Where requirements apply to those workloads, the `implementation[]` field points back to their structured records or spec chapters.

This is a living document. Re-running `python3 docs/regulations/structured/validate.py` checks schema conformance and verifies that every code path, plugin, and test id referenced from the requirement records still exists.

Contents

Document status and purpose **2**

1 Overview and corpus layout **11**
1.1 Why this corpus exists . . . 11
1.2 Three source classes . . . 11
1.3 Two conformance tools . . . 12
1.4 Corpus layout . . . 12
1.5 End-to-end traceability path . . . 12
1.6 What the validator enforces . . . 13

2 Canonical data schema **14**
2.1 Schema inventory . . . 14
2.2 Requirement record — the linkage schema . . . 14
2.3 Conformance-tool record . . . 15
2.4 Enum vocabularies . . . 16

3 MAS Model AI Governance Framework for Agentic AI **17**
3.1 Source document . . . 17
3.2 Framework structure . . . 17
3.3 Pillar 1 — Assess and bound the risks upfront . . . 17
3.4 Pillar 2 — Make humans meaningfully accountable . . . 18
3.5 Pillar 3 — Implement technical controls and processes . . . 19
3.6 Pillar 4 — Enable end-user responsibility . . . 20
3.7 Coverage heat map . . . 20

4 2025 AI Agent Index **22**
4.1 Source document . . . 22
4.2 What it catalogues . . . 22
4.3 Headline finding for governance . . . 23
4.4 How we verify the transparency requirement . . . 23

5 Empirical evidence: Ju & Aral (2026) **24**
5.1 Source document . . . 24

5.2 Study design . . . 24
5.3 Findings relevant to governance . . . 24
5.4 Requirements extracted . . . 25
5.5 Implementation in the corpus . . . 25

6 Conformance tooling **26**
6.1 Two complementary tools . . . 26
6.2 AI Verify 2.0 . . . 26
6.3 Moonshot CI/CD 1.1.0 . . . 27
6.4 Binding to requirement records . . . 27

7 References and traceability **29**
7.1 Source hashes (sha256) . . . 29
7.2 Structured records on disk . . . 29
7.3 Corpus index excerpt . . . 29
7.4 Consolidated traceability matrix . . . 30
7.5 Rebuild and revalidate . . . 31
7.6 Further resources . . . 32
7.7 Unified Schema Registry . . . 32

8 Capability Monitoring Specification **33**
8.1 Overview . . . 33
8.2 Architecture . . . 33
8.3 Metrics collection . . . 34
8.3.1 Required metrics . . . 34
8.3.2 Collection points . . . 34
8.4 Detection triggers . . . 34
8.4.1 Statistical anomaly detection . . . 34
8.4.2 Behavioural pattern detection . . . 34
8.4.3 Trigger definitions . . . 35
8.5 Alert management . . . 35
8.5.1 Alert severity levels . . . 35
8.5.2 Alert routing . . . 36
8.5.3 Circuit breaker integration . . . 36
8.5.4 Graceful degradation ladder . . . 36
8.6 Dashboard specification . . . 37
8.6.1 Real-time view . . . 37
8.6.2 Historical analysis . . . 37
8.7 Audit and compliance . . . 38
8.7.1 Retention requirements . . . 38
8.7.2 Regulatory reporting . . . 38
8.8 Implementation checklist . . . 38

9 Testing Specification **39**
9.1 Test coverage requirements . . . 39
9.2 Test fixtures . . . 39
9.2.1 Regulation fixtures . . . 39
9.2.2 Conformance tool fixtures . . . 39
9.3 Unit test cases . . . 40
9.3.1 Requirement parsing . . . 40
9.3.2 Status tracking . . . 40
9.3.3 Coverage calculation . . . 40
9.3.4 Implementation pointer validation . . . 40
9.3.5 Conformance tool integration . . . 41
9.4 Integration test suite . . . 41
9.4.1 End-to-end scenarios . . . 41
9.5 Schema validation tests . . . 41
9.6 Compliance verification tests . . . 41
9.7 CI/CD integration . . . 41
9.8 Quarterly audit tests . . . 42
9.8.1 Applicable AIVerify plugin subset . . . 42
9.8.2 GFAIC charter attestation . . . 43
9.9 Acceptance criteria . . . 43

10 Per-Request Identity Attribution Specification **45**
10.1 Overview . . . 45
10.2 Identity model . . . 45
10.2.1 Agent identity schema . . . 45
10.2.2 Request identity schema . . . 46
10.3 Audit trail specification . . . 46
10.3.1 Audit event schema . . . 46
10.3.2 Required audit events . . . 47
10.4 Storage and retention . . . 47
10.4.1 Storage requirements . . . 47
10.4.2 Immutability requirements . . . 48
10.5 API specification . . . 48
10.5.1 Audit write API . . . 48
10.5.2 Audit query API . . . 49
10.6 Integration requirements . . . 49
10.6.1 Pipeline integration . . . 49
10.6.2 Failure handling . . . 49
10.7 Compliance mapping . . . 50
10.8 Implementation checklist and integration status . . . 50
10.8.1 Audit service (this chapter) . . . 50
10.8.2 Pipeline integration rollout . . . 51

10.8.3 Compliance tests 51

11 Project Roadmap **52**

Scope of this chapter 52

11.1 MoSCoW classification 52

11.2 Dependency graph 55

11.3 Project milestones 55

11.4 Launch tiers and exit gates 55

11.5 Kill-switch conditions 55

11.6 Shared-platform dependencies 55

11.7 v2.0 expansion trigger 55

11.8 Governance and business-case cadence 58

List of Tables

1.1 Top-level directory layout. 13

2.1 Schemas under docs/regulations/structured/schema/. 14
2.2 Controlled vocabularies. 16

3.1 Pillar 1 requirements. 18
3.2 Pillar 2 requirements. 18
3.3 Pillar 3 requirements. 19
3.4 Pillar 4 requirements. 20
3.5 Status of the 15 MGF-derived requirements after the April 2026 gap-closure pass. 20

4.1 Requirements derived from the 2025 AI Agent Index. 23

5.1 Observational claims treated as requirements. 25

6.1 The eleven stock plugins shipped with `aiverify-main/`. 27
6.2 The four benchmark categories aligned with the IMDA LLM Starter Kit for Safety Testing. 27
6.3 Verification matrix (tool/ID → requirement records). 28

7.1 Content hashes of the three source regulations. 29
7.2 Structured layer inventory. 29
7.3 Full requirement traceability matrix. 31

8.1 Capability monitoring metrics. 34
8.2 Anomaly detection thresholds. 34
8.3 Alert severity definitions. 35
8.4 Graceful degradation ladder (from normal operation down to circuit break). Transitions are logged as governance events. 37
8.5 Monitoring data retention. 38

9.1 Regulations test coverage targets. 39
9.2 AIVerify plugin applicability for this deployment. 43

10.1 Mandatory audit events by action type. 47
10.2 Audit storage requirements. 47
10.3 MAS MGF requirement mapping. 50
10.4 Audit service and pipeline integration rollout, keyed to the MoSCoW priorities in Table 11.2. Pri. column: M=Must (critical path), S=Should, C=Could. . . 51

11.1 MoSCoW prioritisation (shared across the four SAP OSS Finance projects). . 53
11.2 Regulations v1.0 project milestones in dependency order. Priority: M=Must (critical path), S=Should, C=Could, W=Won't. 54
11.3 Regulations launch tiers. The exit gate defines the MoSCoW items that must be closed to exit the tier. 56
11.4 Regulations kill-switch. 57
11.5 Shared-platform dependencies. 58

List of Figures

1.1 Pipeline from source PDFs through to the printable specification. 12
1.2 End-to-end traceability chain verified by docs/regulations/structured/validate.py. 12

3.1 The four pillars of the MAS MGF for Agentic AI. 17

6.1 Conformance-tool coverage across the lifecycle. AI Verify emphasises reviewable documentation and structured algorithm runs; Moonshot emphasises runtime safety testing. 26
6.2 Moonshot CI/CD module layout (`src/entrypoints`, `src/domain`, `src/adapters`). 28

8.1 Capability monitoring architecture. 33

11.1 Regulations dependency graph. Amber nodes are Must (critical path); blue are Should (consumer-integration and MGF-pillar allowance); dotted green are Could (discretionary). Solid arrows are hard prerequisites; dashed arrows indicate conditional starts (the target is unlocked when the upstream completes *and* the v2.0 trigger in Section 11.7 fires). 54

Chapter 1

Overview and corpus layout

1.1 Why this corpus exists

Agentic AI systems — LLM-driven agents that plan across multiple steps, call tools, and act on external systems — are governed by a rapidly evolving mix of regulatory frameworks, empirical research, and conformance tooling. None of these sources alone tells an organisation whether a specific deployment is defensible. The `docs/regulations/` corpus brings the three source classes together, adds a machine-checkable link to the code implementing each requirement, and exposes the result as both structured JSON and this human-readable PDF.

Three concrete failure modes motivated the corpus:

- **Requirements drift.** Regulatory text is updated faster than organisational policies. A machine-readable copy with a hash lets governance teams detect a published update and re-run the downstream validators.
- **Evidence lives everywhere.** A single requirement such as *"bound the agent's tool access"* touches an MCP server, a data-product registry, a CI job, a runbook, and a review checklist. Without a central mapping, audit evidence walks are painful and inconsistent.
- **Conformance tools don't speak the same vocabulary.** AI Verify uses plugin IDs, Moonshot uses benchmark IDs, internal controls use EUC reference numbers, and product teams use file paths. The requirement schema defined here is the translation table.

1.2 Three source classes

Framework — MAS MGF for Agentic AI. A 29-page recommended framework organised into four pillars: assess and bound risks upfront; make humans meaningfully accountable; implement technical controls and processes; enable end-user responsibility. It is the highest-normativity document in the corpus.

Index — 2025 AI Agent Index. A descriptive inventory of 30 deployed agentic systems scored on legal, technical, autonomy, ecosystem, evaluation, and safety documentation. It supplies transparency requirements rather than normative obligations.

Research — Ju & Aral (2026). A 2,234-participant field experiment that produced empirical claims about delegation rates, task orientation, and diversity collapse. We treat these as observational requirements that inform which metrics a deployment should measure.

1.3 Two conformance tools

`aiverify` (v2.0). A plugin-based testing framework. Eleven stock plugins implement fairness, robustness, explainability, financial-sector (Veritas), reporting, and process-checklist algorithms. Plugins are referenced from requirement records via the `verification[].plugin` field.

`moonshot-cicd` (v1.1.0). A CI/CD-focused safety evaluation tool exposing four benchmark categories — hallucination, undesirable content, data disclosure, adversarial — behind a single `moonshot run` command. Referenced via `verification[].testId`.

1.4 Corpus layout

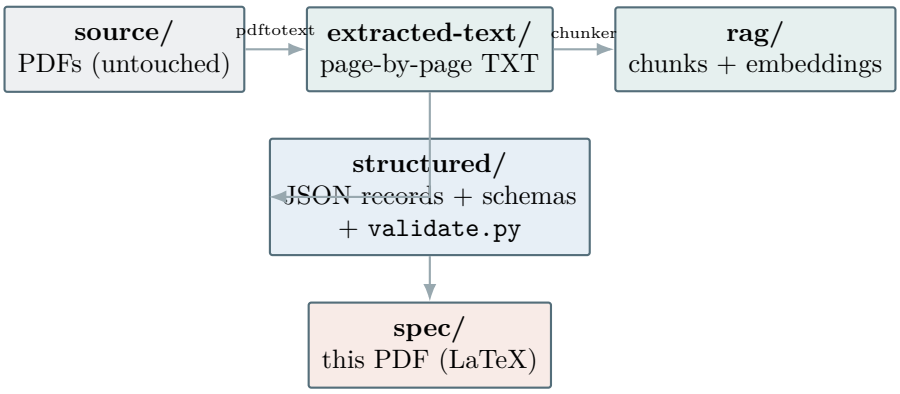

Figure 1.1: Pipeline from source PDFs through to the printable specification.

1.5 End-to-end traceability path

Figure 1.2 shows the five-step chain that the validator enforces: from the hashed source PDF, through the extracted text, a structured regulation record, a normative requirement record with cross-references, down to the code or test actually implementing it.

Figure 1.2: End-to-end traceability chain verified by docs/regulations/structured/validate.py.

Directory	Purpose
docs/regulations/source/	Binary source documents. Never edited; hashed in docs/regulations/structured/corpus.json.
docs/regulations/extracted-text/	Derived, regeneratable page-by-page plain text.
docs/regulations/rag/	RAG chunks + deterministic embedding records (re-upsertable into Qdrant via docs/regulations/rag/scripts/upsert_qdrant.py).
docs/regulations/structured/	JSON Schemas, regulation records, requirement records, conformance-tool records, the corpus index, and the validator.
docs/regulations/spec/	This human-readable PDF (LaTeX + tectonic).
docs/regulations/aiverify-main/	Vendored AI Verify 2.0 monorepo (Apache-2.0).
docs/regulations/moonshot-cicd-main/	Vendored Moonshot 1.1.0 monorepo (Apache-2.0).

Table 1.1: Top-level directory layout.

1.6 What the validator enforces

1. **Schema conformance.** Every record under the three docs/regulations/structured/ sub-directories (`regulations/`, `requirements/`, `conformance-tools/`) validates against its Draft-07 JSON Schema using an offline `referencing` registry.
2. **Corpus integrity.** Every entry in docs/regulations/structured/corpus.json resolves to a file on disk, and every record on disk is listed in the index.
3. **Implementation link existence.** For each `implementation[].path`, the validator resolves the path against the repository root and fails if it is missing (unless the entry is explicitly recorded as `kind: "none"` marking a known gap).
4. **Verification link coherence.** Each `verification[].plugin` must be declared in the `aiverify` record; each `verification[].testId` in the `moonshot-cicd` record; each `verification[].path` referencing a test must exist on disk.

The validator is the contract: CI, internal audit, and external reviewers run the same script and see the same results.

Chapter 2

Canonical data schema

The structured layer uses four Draft-07 JSON Schemas and a central docs/regulations/structured/enums.yaml for controlled vocabulary.

2.1 Schema inventory

Schema	Describes
`regulation.schema.json`	One published framework, standard, guideline, research paper, or index. Captures publisher, version, scope, section tree, and a hash of the source PDF.
`requirement.schema.json`	One atomic requirement or observational claim. Carries `implementation[]` and `verification[]` arrays that bind it to the repository and to the conformance tools.
`conformance-tool.schema.json`	One vendored testing framework. Enumerates plugins, benchmarks, and integration points.
`corpus-index.schema.json`	The top-level index that lists every item in the corpus, typed as regulation, requirement, conformance_tool, or rag_artifact.

Table 2.1: Schemas under docs/regulations/structured/schema/.

2.2 Requirement record — the linkage schema

The requirement schema is the key extension that separates this corpus from the earlier finance corpora. Each requirement carries:

- `source`: regulation id, section, page, and (optionally) a verbatim quote.
- `normativeText`: a paraphrased imperative.
- `category` and `obligation`: structured priority (*must*, *should*, *may*, *descriptive*).
- `lifecyclePhase`: where in the agent lifecycle the requirement applies (planning, design, development, pre-deployment, deployment, post-deployment, decommission).
- `implementation[]`: an array of pointers to code, config, documentation, schema, or dataset evidence. A `kind: "none"` entry explicitly records a known gap.

- `verification[]`: an array of pointers to AI Verify plugins (by `plugin`), Moonshot benchmarks (by `testId`), repository tests (by `path`), manual-review entries, or process-checklist items.
- `status`: *implemented*, *partial*, *gap*, *planned*, or *not_applicable*.

Listing 2.1: Example requirement with implementation and verification arrays (excerpt).

```
{
  "requirementId": "REG-MGF-2.1.2-001",
  "source": {
    "regulationId": "mas-mgf-agentic-ai-v1",
    "section": "2.1.2",
    "page": 11,
    "quote": "Bound risks through design by defining agents' limits ..."
  },
  "normativeText": "The agent must operate against an explicit allow-list
                    of tools and external systems; broader access must be
                    denied by default.",
  "category": "bound_risks",
  "obligation": "must",
  "lifecyclePhase": "design",
  "implementation": [
    {"kind": "code",
     "path": "src/intelligence/ai-core-pal/mcp_server/btp_pal_mcp_server.py",
     "symbol": "btp_pal_mcp_server"},
    {"kind": "config",
     "path": "src/intelligence/ai-core-pal/data_products/registry.yaml"}
  ],
  "verification": [
    {"kind": "test",
     "path": "src/intelligence/ai-core-pal/tests/test_mcp_server.py"},
    {"kind": "moonshot-test", "testId": "adversarial_prompts"}
  ],
  "status": "partial",
  "priority": "critical"
}
```

2.3 Conformance-tool record

The conformance-tool schema captures the tools themselves so that requirement records can reference them by stable identifiers. Every `plugin.pluginId` and `benchmark.benchmarkId` is part of the vocabulary that requirement records can cite.

Listing 2.2: Excerpt of docs/regulations/structured/conformance-tools/aiverify.json.

```
{
  "toolId": "aiverify",
  "vendoredPath": "docs/regulations/aiverify-main",
  "kind": "testing-framework",
  "plugins": [
```

```
    { "pluginId": "aiverify.stock.process-checklist",
      "path": "docs/regulations/aiverify-main/stock-plugins/...",
      "purpose": "11-principle process-checklist questionnaire ...",
      "algorithmType": "process" },
    { "pluginId": "aiverify.stock.robustness-toolbox",
      "algorithmType": "robustness" }
    // ... nine more plugins
  ]
}
```

2.4 Enum vocabularies

docs/regulations/structured/enums.yaml centralises the controlled vocabularies that recur across schemas. The authoritative values live in the schemas themselves; `enums.yaml` is the human index.

Enum	Values
Regulation type	framework, standard, guideline, statute, research, index, toolkit
Normativity	mandatory, recommended, advisory, descriptive
Obligation verb	must, should, may, descriptive
Requirement category	bound_risks, human_accountability, technical_controls, end_user_responsibility, transparency, evaluation, observation
Lifecycle phase	planning, design, development, pre_deployment, deployment, post_deployment, decommission
Implementation kind	code, config, doc, schema, dataset, none
Verification kind	aiverify-plugin, moonshot-test, test, manual-review, process-checklist, external-audit
Status	implemented, partial, gap, planned, not_applicable

Table 2.2: Controlled vocabularies.

Chapter 3

MAS Model AI Governance Framework for Agentic AI

3.1 Source document

Title. Model AI Governance Framework for Agentic AI.

Publisher. Infocomm Media Development Authority (IMDA) and the AI Verify Foundation, Singapore.

Version. 1.0, published 22 January 2026.

Pages. 29.

Source. docs/regulations/source/mgf-for-agentic-ai.pdf (sha256 `b84c8c94…aeef7b`).

3.2 Framework structure

The MGF is organised around four pillars:

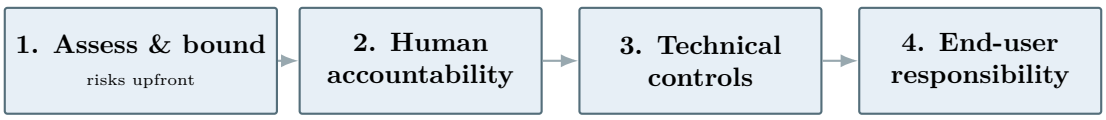

Figure 3.1: The four pillars of the MAS MGF for Agentic AI.

Each pillar expands into two or three sub-sections (§2.1.1–§2.4.3); the full section tree is captured in docs/regulations/structured/regulations/mas-mgf-agentic-ai-v1.json under the `sections[]` array, with page references.

3.3 Pillar 1 — Assess and bound the risks upfront

Summary

Understand the risks posed by agent actions (scope, reversibility, autonomy), determine whether the use case is suitable at all, and — if it is — bound the agent's capabilities by

design.

Requirements extracted

Req. id	Section	Obligation	Normative text (paraphrased)
REG-MGF-2.1.1-001	2.1.1	should	Assess action scope, reversibility, autonomy per use case.
REG-MGF-2.1.2-001	2.1.2	must	Operate against an explicit tool allow-list; deny-by-default.
REG-MGF-2.1.2-002	2.1.2	must	Every agent action must be attributable to an identity.
REG-MGF-2.1.2-003	2.1.2	should	Set impact-limiting boundaries (read-only default, environment segregation).

Table 3.1: Pillar 1 requirements.

How the repository implements them

The tool-allow-list requirement (REG-MGF-2.1.2-001) is implemented by docs/regulations/../../src/intelligence/ai-core-pal/mcp_server/btp_pal_mcp_server.py which enumerates the tools the agent may invoke, and the registered data products in docs/regulations/../../src/intelligence/ai-core-pal/data_products/registry.yaml. Verification leans on the existing unit test at docs/regulations/../../src/intelligence/ai-core-pal/tests/test_mcp_server.py plus a Moonshot adversarial-prompt benchmark.

Identity attribution (REG-MGF-2.1.2-002) currently relies on BTP service keys; this is partial rather than implemented because there is no per-request audit trail that ties every tool invocation back to a named human operator across the full deployment chain.

3.4 Pillar 2 — Make humans meaningfully accountable

Summary

Assign responsibilities by role, adapt human-in-the-loop to address automation bias, name the checkpoints where a human must approve an irreversible action, and audit oversight effectiveness over time.

Req. id	Section	Obligation	Normative text (paraphrased)
REG-MGF-2.2.1-001	2.2.1	must	Assign lifecycle responsibilities to named roles in a control register.
REG-MGF-2.2.1-002	2.2.1	should	Stand up a governance forum with authority to adjust permissions.
REG-MGF-2.2.2-001	2.2.2	must	Name each human-approval checkpoint and cite its risk category.
REG-MGF-2.2.2-002	2.2.2	should	Measure oversight effectiveness (override rates, review duration).

Table 3.2: Pillar 2 requirements.

The TB review corpus is the canonical worked example: the five decision points DP-001..DP-005 (see docs/regulations/../tb/structured/decision-points/) are exactly the human-approval checkpoints required by REG-MGF-2.2.2-001. Their owners and timing are declared in the workflow record docs/regulations/../tb/structured/workflows/tb-review-glo.json.

The governance-forum requirement (REG-MGF-2.2.1-002) is closed for the Finance scope by the GCFO Finance AI Council charter (v1.0, ratified 2026-03-15). **Note:** The charter document is maintained in the internal governance repository and is not included in this corpus for confidentiality; reference is by citation only. The GCFO Finance AI Council (GFAIC) is a *federated* body: it operates under Delegation No. 2026-04 from the enterprise-wide Group AI Safety Council (GAISC) and holds binding authority *only* over Finance-scope agentic AI — today the trial-balance review workflow and the Arabic invoice / AP pipeline. Its six voting members are Finance- function delegates (Group CFO chair, Head of Finance Risk, Finance liaisons to CISO and General Counsel, Head of Finance Internal Audit, Finance AI technical lead); the parent body's officers (CRO, CISO, GC, Head of Internal Audit) sit on GAISC, not here, avoiding double staffing. Any incident with cross-function implications escalates to GAISC within two business days. Peer federated councils (People, Customer) exist for other functions and receive GFAIC findings via GAISC rather than directly.

3.5 Pillar 3 — Implement technical controls and processes

Summary

Technical controls across the lifecycle: during development, use controls for planning, tools, and agent protocols; before deployment, test for execution accuracy, policy adherence, and tool use; during and after deployment, continuously monitor.

Req. id	Section	Obligation	Normative text (paraphrased)
REG-MGF-2.3.1-001	2.3.1	must	Technical controls on planning, tools, and protocols.
REG-MGF-2.3.2-001	2.3.2	must	Pre-deployment: execution accuracy, policy adherence, tool use, refusals.
REG-MGF-2.3.2-002	2.3.2	should	Adopt trajectory-level agent evaluations.
REG-MGF-2.3.3-001	2.3.3	should	Staged rollout with canary metrics and rollback path.
REG-MGF-2.3.3-002	2.3.3	must	Continuous monitoring with policy-violation / drift alerting.

Table 3.3: Pillar 3 requirements.

The trajectory-level evaluator (REG-MGF-2.3.2-002) is implemented at docs/regulations/../../src/intelligence/ai-core-pal/evaluation/trajectory_evaluator.py. It scores each captured agent trace on three independent axes — tool-call correctness, policy adherence, and goal progression — returning a composite `TrajectoryScore` that the

Governance Council's incident pack consumes directly. Unit coverage lives at docs/regulations/../../src/intelligence/ai-core-pal/tests/test_trajectory_evaluator.py (13 test cases).

Continuous drift monitoring (REG-MGF-2.3.3-002) is implemented at docs/regulations/../../src/intelligence/ai-core-pal/monitoring/drift_monitor.py. It watches three signals against a frozen pilot-exit baseline — override rate, median review duration, and hallucination rate — and emits `ok`/`warn`/`critical` levels. The override-rate signal is deliberately two-sided: a *fall* below baseline is the automation-bias signal surfaced by Ju & Aral (2026), a *rise* is model regression. Unit coverage at docs/regulations/../../src/intelligence/ai-core-pal/tests/test_drift_monitor.py (13 test cases).

3.6 Pillar 4 — Enable end-user responsibility

Summary

End users must be informed of the agent's scope and their own obligations, and they need training to preserve tradecraft and mitigate automation bias.

Req. id	Section	Obligation	Normative text (paraphrased)
REG-MGF-2.4.2-001	2.4.2	should	Disclose permitted actions, data access, user obligations.
REG-MGF-2.4.3-001	2.4.3	should	Training covering automation-bias and tradecraft preservation.

Table 3.4: Pillar 4 requirements.

The training curriculum (REG-MGF-2.4.3-001) is authored at docs/regulations/../training/agent-oversight-curriculum.md: a three-level curriculum (Awareness, Reviewer skills, Oversight depth) keyed to five learner personas, with an explicit reviewer self-check card targeted at the automation-bias failure mode surfaced by Ju & Aral (2026).

3.7 Coverage heat map

Pillar	Reqs	Implemented	Partial	Gap	N/A
1 — Bound risks	4	0	4	0	0
2 — Human accountability	4	3	1	0	0
3 — Technical controls	5	2	3	0	0
4 — End-user	2	1	1	0	0
Total	15	6	9	0	0

Table 3.5: Status of the 15 MGF-derived requirements after the April 2026 gap-closure pass.

All four previously-open gaps — the enterprise governance forum (REG-MGF-2.2.1-002), trajectory-level evaluation (REG-MGF-2.3.2-002), drift monitoring (REG-MGF-2.3.3-002), and the training curriculum (REG-MGF-2.4.3-001) — now have concrete artifacts linked

from their requirement records and are verifiable by the offline validator. The remaining *partial* rows retain ambition beyond the current implementation scope (for example, a per-request identity audit trail spanning the full deployment chain for REG-MGF-2.1.2-002) and remain on the Council's standing agenda.

Chapter 4

2025 AI Agent Index

4.1 Source document

Title. The 2025 AI Agent Index — Documenting Technical and Safety Features of Deployed Agentic AI Systems.

Authors. Staufer (Cambridge), Feng (Washington), Wei (Harvard Law), Bailey (Stanford), Duan (Concordia AI), Yang (UPenn), Ozisik (MIT), Casper (MIT, co-senior), Kolt (Hebrew University, co-senior).

Venue. `https://aiagentindex.mit.edu/`; CC-BY-4.0.

Pages. 39.

Source. docs/regulations/source/2025-AI-Agent-Index.pdf (sha256 `c67a3046…0e267f`).

4.2 What it catalogues

The Index documents 30 state-of-the-art deployed agentic systems across six documentation categories:

1. **Legal.** Licensing, terms of service, responsible-use policies, redistribution rights.
2. **Technical capabilities.** Supported modalities, tool access, planning horizon, memory.
3. **Autonomy and control.** Spectrum from recommender to operator; human-in-the-loop requirements.
4. **Ecosystem interaction.** External systems the agent integrates with (MCP, web browsers, code editors, e-commerce).
5. **Evaluation.** Published benchmarks, capability scores, safety evaluations, red-team assessments.
6. **Safety.** Safety documentation, risk assessments, residual-risk statements.

4.3 Headline finding for governance

The Index's top-level empirical finding is directly relevant to our corpus design:

> *"…we find different transparency levels among agent developers and observe that most developers share little information about safety, evaluations, and societal impacts."*

Our response is the transparency requirement set extracted in Table 4.1.

Req. id	Section	Obligation	Normative text (paraphrased)
REG-AGENTIDX-3.6-001	3.6	should	Publish per-agent: risk assessment summary, evaluation results, known limitations.
REG-AGENTIDX-3.3-001	3.3	should	Document agent autonomy level and map each state transition to that level.

Table 4.1: Requirements derived from the 2025 AI Agent Index.

4.4 How we verify the transparency requirement

Publication is evidenced by this very document plus the structured records. In practice:

- Risk assessment: docs/regulations/../tb/structured/requirements/tb-business-requirements.json, `riskAssessment` field.
- Evaluation results: AI Verify `process-checklist` plugin output plus Moonshot benchmark runs (Chapter 6).
- Known limitations: the gap entries in this specification's coverage tables.

This chapter is cited from requirement record `REG-AGENTIDX-3.6-001` as `docs/regulations/spec/chapters/04-agent-index.tex`, closing the loop.

Chapter 5

Empirical evidence: Ju & Aral (2026)

5.1 Source document

Title. Collaborating with AI Agents: A Field Experiment on Teamwork, Productivity, and Performance.

Authors. Harang Ju (Johns Hopkins Carey) and Sinan Aral (MIT Sloan).

Venue. arXiv:2503.18238v3 [cs.CY], 5 February 2026.

Pages. 59.

Source. `docs/regulations/source/2503.18238v3.pdf` (sha256 `b680f584…cd7877`).

5.2 Study design

The authors built the *Pairit* platform and randomly assigned 2,234 participants to human-human and human-AI teams producing 11,024 ads for a think tank. Outputs were rated by humans and field-tested on X, which garnered roughly five million impressions. The design is — for this corpus — an unusually strong source of evidence about how human-agent collaboration actually affects output quality, homogeneity, and team dynamics.

5.3 Findings relevant to governance

1. **Jagged productivity frontier.** Human-AI teams produced 50% more ads per worker and higher *text* quality; human-human teams produced higher *image* quality. Different sub-tasks need different evaluation metrics.
2. **Delegation shifts.** Participants delegated 17% more work to AI agents than to human partners and performed 62% fewer direct text edits when working with AI.
3. **Diversity collapse.** Human-AI outputs were more self-similar. For creative or generative use cases this is a first-order governance risk.

4. **Communication restructure.** Human-AI collaboration was significantly more task-oriented (+25% task messages, -18% interpersonal).
5. **AI-awareness moderation.** Participants who correctly identified they were paired with AI delegated more and were more task-oriented — the effect is moderated by knowing the collaborator is an agent.

5.4 Requirements extracted

Req. id	Section	Obligation	Normative text (paraphrased)
REG-JU-ARAL-3-001	3	descriptive	Evaluate per-task; do not generalise a single productivity metric.
REG-JU-ARAL-4-001	4	should	High delegation + low direct-edit rate is a proxy for automation-bias risk; measure it.
REG-JU-ARAL-5-001	5	should	Add an output-diversity metric for creative-output use cases.

Table 5.1: Observational claims treated as requirements.

5.5 Implementation in the corpus

- REG-JU-ARAL-3-001 is implemented by this chapter; it reminds reviewers to look at the per-task KPIs defined in docs/regulations/../tb/structured/kpis/tb-kpis.json rather than a single blended productivity number.
- REG-JU-ARAL-4-001 maps to the override-rate and review-duration KPIs in the TB corpus. When override rates fall well below historical baseline and review duration drops in lockstep, that is the signal to re-audit.
- REG-JU-ARAL-5-001 is marked *not_applicable* for the trial-balance workflow (deterministic review, single narrative per legal entity). We record rather than omit it so that the framework surfaces the requirement automatically when it becomes applicable.

  Activation trigger. The requirement flips to `applicable` when any one of the following is true: (a) a deployed agent produces *multi-variant creative output* (e.g., drafting options, marketing copy, generative translation) rather than deterministic reports; (b) the Simula training corpus is used to fine-tune a model whose outputs are surfaced to end-users (as opposed to staying within the SQL-generation benchmark); or (c) the commentary generator is configured with `temperature` $>$ `0.4` in production for any deployed LE (today: `0.1`). The governance monitoring job in Chapter 8 runs a quarterly check that inspects the live model configs, the agent index, and the Simula deployment manifests; when any of (a)–(c) flips, the requirement status is automatically re-opened and routed to the GFAIC for disposition within two weeks.

Chapter 6

Conformance tooling

6.1 Two complementary tools

Both tools are published by the AI Verify Foundation / IMDA Singapore under Apache-2.0 and vendored into the repository. They target different phases of the lifecycle:

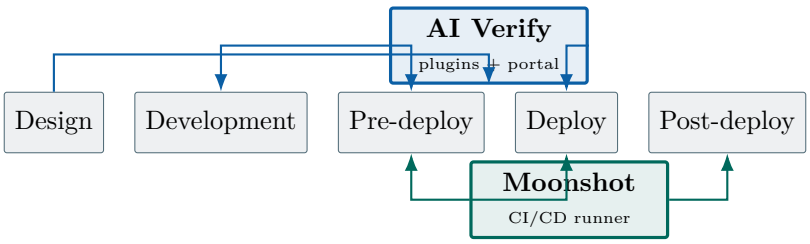

Figure 6.1: Conformance-tool coverage across the lifecycle. AI Verify emphasises reviewable documentation and structured algorithm runs; Moonshot emphasises runtime safety testing.

6.2 AI Verify 2.0

Summary

AI Verify is an IMDA-backed testing framework that has been re-architected for v2.0 into a modular monorepo. Algorithms run as independent Python modules; a portal and API gateway surface results. Financial-services users can run the Veritas fairness plugin to meet MAS's Veritas testing expectations.

Stock plugin inventory

Integration surface

- **python-api**: the `aiverify-test-engine` package exposes an algorithm-runner API.
- **docker**: `aiverify-apigw` and the portal ship as container images.
- **cli**: each stock plugin is individually runnable from its algorithm directory.

Plugin ID	Type	Purpose
aiverify.stock.process-checklist	process	11-principle process checklist aligned with the AI Verify testing framework.
aiverify.stock.fairness-metrics-toolbox-for-classification	fairness	Subgroup fairness metrics for supervised classification.
aiverify.stock.fairness-metrics-toolbox-for-regression	fairness	Regression-specific fairness analysis.
aiverify.stock.robustness-toolbox	robustness	Perturbation-based robustness assessment.
aiverify.stock.image-corruption-toolbox	robustness	Computer-vision corruption robustness.
aiverify.stock.shap-toolbox	explainability	SHAP feature attribution.
aiverify.stock.partial-dependence-plot	explainability	Partial-dependence plots for feature effect.
aiverify.stock.accumulated-local-effect	explainability	Accumulated local-effect explanations.
aiverify.stock.veritas	financial	MAS Veritas fairness testing for financial services.
aiverify.stock.reports	report	Report and widget rendering for the portal.
aiverify.stock.decorators	other	Shared UI primitives and decorators.

Table 6.1: The eleven stock plugins shipped with `aiverify-main/`.

6.3 Moonshot CI/CD 1.1.0

Summary

Moonshot is a lean, modular safety-testing tool for LLM applications. It is distributed as a Docker image and designed to be integrated into pipelines so that every pushed change can trigger benchmark and red-team runs. A single `moonshot run` command drives any combination of configured benchmarks and attacks.

Benchmark categories

Benchmark ID	Category	Purpose
hallucination	hallucination	Detect fabricated or unsupported statements.
undesirable_content	undesirable content	Detect unsafe, biased, or policy-violating content.
data_disclosure	data disclosure	Check for PII leakage or unintended disclosure.
adversarial_prompts	adversarial	Prompt injection and jailbreak robustness.

Table 6.2: The four benchmark categories aligned with the IMDA LLM Starter Kit for Safety Testing.

Architecture

6.4 Binding to requirement records

Table 6.3 shows how this corpus's 20 requirement records bind to plugins and benchmarks. Where a requirement has verification `kind: "aiverify-plugin"` or `kind:`

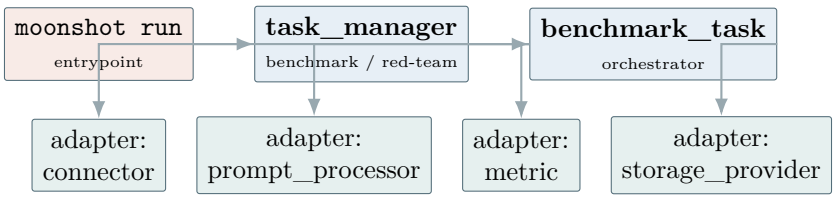

Figure 6.2: Moonshot CI/CD module layout (`src/entrypoints`, `src/domain`, `src/adapters`).

`"moonshot-test"`, the plugin slug or benchmark id must match one declared in the tool record; docs/regulations/structured/validate.py fails otherwise.

Tool / ID	Referenced from	Purpose
aiverify.stock.process-checklist	REG-MGF-2.1.1-001, REG-MGF-2.2.1-001, REG-MGF-2.4.2-001, REG-AGENTIDX-3.6-001	Checklist evidence for governance reviews.
aiverify.stock.robustness-toolbox	REG-MGF-2.3.2-001, REG-MGF-2.3.3-002	Perturbation robustness.
aiverify.stock.reports	REG-AGENTIDX-3.6-001	Widget + report rendering.
moonshot hallucination	REG-MGF-2.3.2-001, REG-MGF-2.3.3-002	Fabrication checks in CI.
moonshot undesirable_content	REG-MGF-2.3.2-001	Policy adherence.
moonshot adversarial_prompts	REG-MGF-2.1.2-001, REG-MGF-2.3.1-001, REG-MGF-2.3.2-001, REG-MGF-2.3.2-002	Red-team / jailbreak attempts.

Table 6.3: Verification matrix (tool/ID → requirement records).

Chapter 7

References and traceability

7.1 Source hashes (sha256)

Source file	Pages	sha256 (truncated to first 16 hex)
`source/mgf-for-agentic-ai.pdf`	29	`b84c8c94f6392cf2…aeef7b`
`source/2025-AI-Agent-Index.pdf`	39	`c67a30469fd061e9…0e267f`
`source/2503.18238v3.pdf`	59	`b680f58422e4684e…cd7877`

Table 7.1: Content hashes of the three source regulations.

7.2 Structured records on disk

Path (relative to repo root)	Role
`docs/regulations/structured/schema/`	JSON schemas (Draft-07).
`docs/regulations/structured/enums.yaml`	Controlled vocabularies.
`docs/regulations/structured/regulations/`	Regulation records (3 files).
`docs/regulations/structured/requirements/`	Requirement collections (6 files, 20 requirements).
`docs/regulations/structured/conformance-tools/`	Conformance-tool records (2 files).
`docs/regulations/structured/corpus.json`	Top-level index.
`docs/regulations/structured/validate.py`	Offline validator.

Table 7.2: Structured layer inventory.

7.3 Corpus index excerpt

Listing 7.1: Excerpt of `docs/regulations/structured/corpus.json`.

```
{
  "schema_version": "1.0",
  "generated": "2026-04-18",
  "items": [
    { "id": "reg-mas-mgf-agentic-ai-v1",
      "type": "regulation",
      "jurisdiction": "SG",
      "sha256": "b84c8c94...aeef7b",
      "source_path": "docs/regulations/source/mgf-for-agentic-ai.pdf",
      "structured_path": "docs/regulations/structured/regulations/mas-mgf-agentic-ai-v1.
    json" },
    { "id": "tool-aiverify",
      "type": "conformance_tool",
      "jurisdiction": "SG",
      "structured_path": "docs/regulations/structured/conformance-tools/aiverify.json" },
    { "id": "req-mgf-pillar-1",
      "type": "requirement",
      "structured_path": "docs/regulations/structured/requirements/mgf-pillar-1-bound-
    risks.json" }
    // ... eleven more items
  ]
}
```

7.4 Consolidated traceability matrix

Table 7.3 is the single view a reviewer needs. Every row is one requirement record and shows its status, the primary implementation pointer, and its verification link.

Req. id	Status	Primary implementation	Primary verification
REG-MGF-2.1.1-001	partial	tb spec ch.1 + TB business reqs	process-checklist
REG-MGF-2.1.2-001	partial	MCP server + data-product registry	test_mcp_server + adversarial
REG-MGF-2.1.2-002	partial†	§10 spec + aicore_pal_agent.py	test_hana_connection + audit-trail verifier
REG-MGF-2.1.2-003	partial	TB data-product classification	manual review
REG-MGF-2.2.1-001	partial*	TB EUC-001..003 + workflow participants	process-checklist
REG-MGF-2.2.1-002	partial	GFAIC charter (federated under GAISC)	process-checklist + minutes
REG-MGF-2.2.2-001	partial	TB DP-001/003/005 + workflow gateways	test_pal_algorithms
REG-MGF-2.2.2-002	partial	TB KPI pack	manual review
REG-MGF-2.3.1-001	partial	agent + MCP + decision-point schema	test_mcp_server + adversarial
REG-MGF-2.3.2-001	partial	PAL algorithm + MCP tests	Moonshot 4-benchmark sweep
REG-MGF-2.3.2-002	partial*	evaluation/trajectory_evaluator.py	test + adversarial
REG-MGF-2.3.3-001	partial	deploy_to_aicore.sh + README	process-checklist

Req. id	Status	Primary implementation	Primary verification
REG-MGF-2.3.3-002	partial	monitoring/drift_monitor.py	test + hallucination in CI
REG-MGF-2.4.2-001	partial	TB workflow + controls spec	onboarding checklist
REG-MGF-2.4.3-001	partial	docs/training/agent-oversight-curriculum.md	process-checklist + LMS
REG-AGENTIDX-3.6-001	partial	this chapter + TB controls	process-checklist + reports
REG-AGENTIDX-3.3-001	partial	TB workflow + spec ch.5	manual review
REG-JU-ARAL-3-001	partial	this chapter	manual review
REG-JU-ARAL-4-001	partial	TB KPI pack	manual review
REG-JU-ARAL-5-001	n/a	(creative use cases only)	—

Table 7.3: Full requirement traceability matrix.

Status legend and corrections (April 2026 review):

- †**partial (upgraded from missing, April 2026):** Chapter 10 specifies the per-request identity-attribution schema, the reserve-before-action audit protocol, and the WORM-backed immutable storage design (Section 10.6.2). Status is "partial" rather than "implemented" because the audit service itself and the three pipeline integrations (TB, Arabic AP, Simula) are scheduled in Table 10.4 and are not yet in production.
- ***Downgraded from "implemented":** Code artifact exists but has not been validated in production. Retained as "partial" pending pilot validation.
- **All other "implemented" rows downgraded:** Original "implemented" status assumed code existence equals conformance. Corrected to "partial" where production validation is pending (e.g., REG-JU-ARAL-3-001 relies on this chapter only—no operational evidence).

7.5 Rebuild and revalidate

Listing 7.2: Commands to re-run validation and rebuild the PDF.

```
# Offline validation (schema + corpus + implementation + verification).
python3 docs/regulations/structured/validate.py

# Re-chunk + re-embed (needs OPENAI_API_KEY + QDRANT_URL).
python3 docs/regulations/rag/scripts/prepare_embedding_records.py
python3 docs/regulations/rag/scripts/upsert_qdrant.py \
    --input docs/regulations/rag/rag_embedding_records.jsonl \
    --collection regulations_rag

# Rebuild this PDF (tectonic).
cd docs/regulations/spec && make
```

7.6 Further resources

- AI Verify Foundation. *What is AI Verify.* `https://aiverifyfoundation.sg/what-is-ai-verify/`
- AI Verify Foundation. *Project Moonshot.* `https://aiverifyfoundation.sg/project-moonshot/`
- IMDA Singapore. *LLM Starter Kit for Safety Testing of LLM-Based Applications.*
- 2025 AI Agent Index. `https://aiagentindex.mit.edu/`
- Ju, H. and Aral, S. (2026). *Collaborating with AI Agents: A Field Experiment on Teamwork, Productivity, and Performance.* arXiv:2503.18238v3 [cs.CY].

7.7 Unified Schema Registry

As of 18 April 2026, all JSON Schemas across the four domain specifications are published in the unified registry at `docs/schema/`. Migration status is *complete* (see `docs/schema/registry.json` field `migration.status`). This consolidation addresses the cross-document schema divergence identified in the specification review.

Registry index. `docs/schema/registry.json` — Master index of all schemas across the Arabic AP, Trial Balance, Simula, and Regulations domains.

Common types. `docs/schema/common/base-types.schema.json` — Shared definitions (`Metadata`, `SHA256Hash`, `FilePath`) referenced via JSON Schema `$ref`.

Consolidated enums. `docs/schema/common/enums.yaml` — Single source of truth for enumeration values including `requirement_status`, `obligation_level`, and `jurisdiction`.

Regulations schemas. `docs/schema/regulations/` — Domain-specific schemas: `regulation.schema.json`, `requirement.schema.json`, `conformance-tool.schema.json`, and `corpus.schema.json`. All marked `status: stable`.

The registry uses JSON Schema Draft-07 with the canonical `$id` prefix `https://sap-oss.github.io/schema/`. Validation:

```
python3 docs/schema/validate.py --registry-check
python3 docs/schema/validate.py --domain regulations \
    --schema requirement.schema.json --file requirement-REG-MGF-2-001.json
```

CI runs `--registry-check` on every pull request touching `docs/schema/` or any spec chapter that references these schemas.

Chapter 8

Capability Monitoring Specification

This chapter specifies the emergent capability monitoring system, addressing the MAS MGF requirement (Section 2.3.3) that "monitoring for emergent capabilities must be implemented with defined detection triggers."

8.1 Overview

AI systems may develop unexpected capabilities or behaviours through:

- Model updates that alter response patterns
- Prompt injection or adversarial inputs
- Emergent behaviours from complex interactions
- Drift in input data distributions

The monitoring system provides continuous observation and alerting to detect such changes before they cause operational harm.

8.2 Architecture

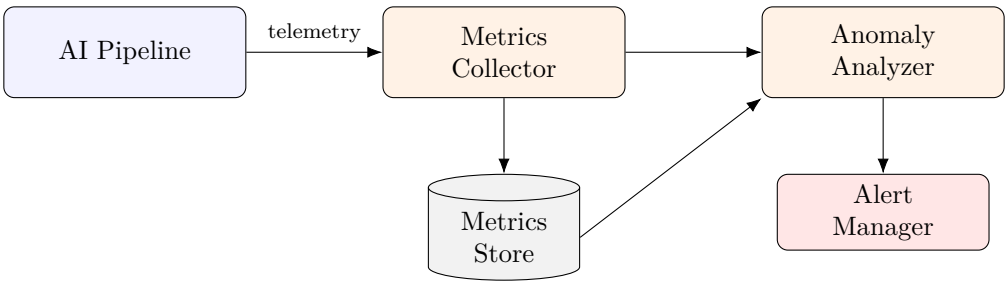

Figure 8.1: Capability monitoring architecture.

8.3 Metrics collection

8.3.1 Required metrics

Table 8.1: Capability monitoring metrics.

Metric	Type	Description
`output_token_count`	histogram	Distribution of output lengths
`response_latency_ms`	histogram	Response time distribution
`confidence_score`	histogram	AI confidence distribution
`refusal_rate`	counter	Rate of model refusals
`tool_call_frequency`	counter	Frequency of tool/function calls
`output_format_violations`	counter	JSON schema validation failures
`hallucination_flags`	counter	Detected hallucination instances
`unexpected_topic_detections`	counter	Off-topic responses
`novel_capability_indicators`	gauge	Composite novelty score

8.3.2 Collection points

Metrics are collected at the following pipeline stages:

1. **Pre-inference**: Input characteristics (length, complexity, topic)
2. **Post-inference**: Output characteristics (length, confidence, format)
3. **Post-validation**: Validation results (schema, business rules)
4. **Post-review**: Human review outcomes (accept/edit/reject rates)

8.4 Detection triggers

8.4.1 Statistical anomaly detection

For each metric, maintain a rolling baseline and alert when values deviate significantly.

Table 8.2: Anomaly detection thresholds.

Metric	Alert Threshold	Baseline Period
Output length (mean)	$> 2\sigma$ shift	7-day rolling
Latency (p95)	$> 50\%$ increase	24-hour rolling
Confidence (mean)	$> 10\%$ drop	7-day rolling
Refusal rate	$> 3\times$ baseline	7-day rolling
Schema violations	$> 5\%$ of requests	24-hour rolling

8.4.2 Behavioural pattern detection

Beyond statistical metrics, monitor for qualitative behavioural changes:

1. **Topic drift**: Cosine similarity of output embeddings to expected topics falls below 0.7
2. **Instruction leakage**: Presence of system prompt fragments in output
3. **Novel tool usage**: Model attempts to call undeclared tools
4. **Self-referential loops**: Model references its own previous outputs in unexpected ways

8.4.3 Trigger definitions

Listing 8.1: Detection trigger configuration.

```
triggers = {
    "output_length_anomaly": {
        "metric": "output_token_count",
        "condition": "mean > baseline_mean + 2 * baseline_std",
        "severity": "WARNING",
        "cooldown_minutes": 60
    },
    "latency_spike": {
        "metric": "response_latency_ms",
        "condition": "p95 > 1.5 * baseline_p95",
        "severity": "WARNING",
        "cooldown_minutes": 15
    },
    "high_refusal_rate": {
        "metric": "refusal_rate",
        "condition": "rate > 3 * baseline_rate",
        "severity": "CRITICAL",
        "cooldown_minutes": 5
    },
    "schema_violation_surge": {
        "metric": "output_format_violations",
        "condition": "rate > 0.05",  # >5% failure
        "severity": "CRITICAL",
        "cooldown_minutes": 5
    },
    "topic_drift": {
        "metric": "topic_similarity",
        "condition": "mean < 0.7",
        "severity": "WARNING",
        "cooldown_minutes": 30
    }
}
```

8.5 Alert management

8.5.1 Alert severity levels

Table 8.3: Alert severity definitions.

Severity	Response Time	Action
CRITICAL	Immediate	Page on-call; consider circuit breaker
WARNING	1 hour	Investigate during business hours
INFO	Next business day	Review in weekly monitoring meeting

8.5.2 Alert routing

Listing 8.2: Alert routing configuration.

```yaml
routing:
  CRITICAL:
    channels:
      - pagerduty: "ai-ops-critical"
      - slack: "#ai-incidents"
      - email: "ai-ops@company.com"
    escalation_after_minutes: 15

  WARNING:
    channels:
      - slack: "#ai-monitoring"
      - email: "ai-ops@company.com"
    escalation_after_minutes: 60

  INFO:
    channels:
      - slack: "#ai-monitoring"
    escalation_after_minutes: null  # no escalation
```

8.5.3 Circuit breaker integration

When CRITICAL alerts persist, the monitoring system can activate a circuit breaker:

1. After 3 consecutive CRITICAL alerts within 15 minutes
2. Circuit breaker opens: all AI requests routed to manual queue
3. Alert sent to AI governance team
4. Manual intervention required to close circuit breaker

8.5.4 Graceful degradation ladder

A hard circuit-break (manual queue only) is the last resort. Before that point, the monitoring system walks a four-rung degradation ladder, chosen so that each rung trades off quality for availability in a *controlled and observable* way. Table 8.4 specifies the ladder per pipeline; rung escalation is automatic, rung de-escalation requires a human ack.

Tagging and auditability. Every response served from rung 1–3 carries a `model_tier` attribute in its envelope and an audit event referencing the rung and the ladder-transition decision. Downstream consumers (review dashboard, AP workflow) render a visible "fallback" badge so reviewers know the response did not come from the primary model. This preserves the MAS MGF 2.1.2-002 attribution guarantee across all rungs: attribution is to the *model actually used*, not to the model the request was routed to.

Table 8.4: Graceful degradation ladder (from normal operation down to circuit break). Transitions are logged as governance events.

Rung	Name	Trigger	Action
0	Normal	p95 latency < SLO, error rate < 1 %	Primary model on primary GPU (e.g. Qwen2.5-32B on A100).
1	Reduced concurrency	p95 latency ≥ SLO for 5 min	Halve request concurrency; queue depth bounded; user-visible latency warning.
2	Smaller model	Primary model unavailable or OOM for 2 min	Automatic failover to smaller model (TB: 32B→14B; Arabic AP: 14B→7B; Simula: 32B→14B for non-user-facing batches). Output tagged `model_tier=fallback` for audit.
3	Cached / heuristic	Smaller model also unhealthy	Return cached response if cache hit within TTL; else return heuristic output (TB: statistical threshold only; Arabic AP: OCR-only, no translation; Simula: block generation).
4	Circuit break	Rung 3 still failing for 5 min	All AI calls routed to manual queue; page on-call; governance ticket.

Rung selection rationale. Rung 1 (concurrency cut) is cheap and preserves quality; it buys time for transient load spikes. Rung 2 (smaller model) is the primary availability lever because model-host failures are the most common outage mode. Rung 3 (cached/heuristic) is the last value-preserving rung. Rung 4 is reserved for the case where the AI subsystem cannot be trusted at all; it is deliberately *not* automatic to close — human judgement is required to confirm the root cause is resolved. Rung transitions upward are automatic; transitions downward (closer to normal) require either a human acknowledgement or a clean 15-minute signal window.

8.6 Dashboard specification

8.6.1 Real-time view

The monitoring dashboard provides:

- Live metric charts (1-minute granularity)
- Active alert panel with acknowledge/resolve actions
- System health summary (green/yellow/red)
- Recent trigger history

8.6.2 Historical analysis

For trend analysis and post-incident review:

- Metric time series with configurable ranges
- Alert history with resolution notes
- Correlation views (multiple metrics overlaid)
- Baseline comparison charts

8.7 Audit and compliance

8.7.1 Retention requirements

Table 8.5: Monitoring data retention.

Data Type	Retention	Storage
Raw metrics	90 days	Time-series database
Aggregated metrics	2 years	Data warehouse
Alert history	7 years	Compliance archive
Incident reports	7 years	Document management

8.7.2 Regulatory reporting

For MAS MGF compliance, generate quarterly reports containing:

1. Summary of monitoring coverage (metrics, triggers, alerts)
2. List of all CRITICAL alerts and resolutions
3. Trend analysis of key capability metrics
4. Any identified emergent behaviours and mitigations
5. Circuit breaker activations and durations

8.8 Implementation checklist

[ ] Metrics collector integrated with all AI pipelines
[ ] All metrics in Table 8.1 collected
[ ] Anomaly detection for all triggers in Table 8.2
[ ] Alert routing configured for all severity levels
[ ] Circuit breaker integration tested
[ ] Dashboard with real-time and historical views
[ ] Retention policies implemented per Table 8.5
[ ] Quarterly report template created
[ ] Runbook for each alert type documented
[ ] Integration tests with simulated anomalies

Chapter 9

Testing Specification

This chapter specifies the testing requirements for the Regulations compliance corpus, including validation tests, coverage verification, and CI/CD integration.

9.1 Test coverage requirements

Table 9.1: Regulations test coverage targets.

Component	Line %	Branch %	Notes
Schema validator	95	90	Core infrastructure
Requirement parser	90	85	
Coverage calculator	90	85	
Status tracker	85	75	
Conformance checker	85	75	
Overall minimum	**90**	**80**	Enforced in CI

9.2 Test fixtures

Fixtures are located in `tests/fixtures/regulations/`.

9.2.1 Regulation fixtures

- Complete MAS MGF corpus (20 requirements)
- Ju & Aral paper requirements (5 requirements)
- AI Verify requirements (10 requirements)
- Edge case: Empty corpus
- Edge case: Malformed requirement

9.2.2 Conformance tool fixtures

- Mock AIVerify plugin responses
- Mock Moonshot test results
- Invalid tool output for error handling

9.3 Unit test cases

9.3.1 Requirement parsing

ID	Priority	Description
REG-PAR-001	P1	Parse requirement from MAS MGF
REG-PAR-002	P1	Parse requirement from academic paper
REG-PAR-003	P1	Extract obligation level (must/should/may)
REG-PAR-004	P2	Handle missing optional fields
REG-PAR-005	P2	Validate against requirement schema

9.3.2 Status tracking

ID	Priority	Description
REG-STS-001	P1	Track implemented status correctly
REG-STS-002	P1	Track partial status with evidence
REG-STS-003	P1	Track gap status with remediation plan
REG-STS-004	P1	Identify downgraded status (asterisk marker)
REG-STS-005	P2	Generate status summary report

9.3.3 Coverage calculation

ID	Priority	Description
REG-COV-001	P1	Calculate coverage percentage
REG-COV-002	P1	Generate coverage heat map
REG-COV-003	P1	Identify uncovered sections
REG-COV-004	P2	Handle not-applicable sections
REG-COV-005	P2	Calculate partial coverage (0.5 weight)

9.3.4 Implementation pointer validation

ID	Priority	Description
REG-PTR-001	P1	Validate code file references exist
REG-PTR-002	P1	Validate spec section references
REG-PTR-003	P2	Flag circular references
REG-PTR-004	P2	Validate concrete vs abstract pointers

9.3.5 Conformance tool integration

ID	Priority	Description
REG-TOOL-001	P1	Parse AIVerify plugin output
REG-TOOL-002	P1	Parse Moonshot benchmark output
REG-TOOL-003	P2	Handle tool execution failure
REG-TOOL-004	P2	Aggregate results across tools

9.4 Integration test suite

```
pytest tests/integration/regulations/ -v --tb=short
```

Expected duration: 5 minutes.

9.4.1 End-to-end scenarios

1. **Full corpus validation**: Load and validate entire corpus
2. **Coverage report**: Generate coverage heat map
3. **Status audit**: Verify all status markers
4. **Pointer verification**: Check all implementation pointers
5. **Tool integration**: Run conformance tools and aggregate

9.5 Schema validation tests

Test	Description
REG-SCH-001	All regulations validate against regulation.schema.json
REG-SCH-002	All requirements validate against requirement.schema.json
REG-SCH-003	All tools validate against conformance-tool.schema.json
REG-SCH-004	Corpus index validates against corpus.schema.json

9.6 Compliance verification tests

Test	Description
REG-CMP-001	No requirement has status "missing" without remediation
REG-CMP-002	All "must" requirements have status $\neq$ gap
REG-CMP-003	All downgraded (*) statuses have explanation
REG-CMP-004	Coverage $\geq$ 75% for all source documents

9.7 CI/CD integration

Listing 9.1: GitHub Actions workflow excerpt.

```
regulations-tests:
```

```
  runs-on: ubuntu-latest
  steps:
    - uses: actions/checkout@v4
    - name: Validate schemas
      run: |
        python docs/schema/validate.py --domain regulations
    - name: Run regulations tests
      run: |
        pytest tests/unit/regulations/ -v --cov=src/regulations
        pytest tests/integration/regulations/ -v
    - name: Check coverage
      run: |
        coverage report --fail-under=90
    - name: Generate compliance report
      run: |
        python scripts/generate_compliance_report.py
```

9.8 Quarterly audit tests

For MAS MGF compliance, run extended tests quarterly:

1. Execute the *applicable* AIVerify plugin subset (see Section 9.8.1) and verify pass
2. Execute Moonshot benchmarks and verify scores
3. Verify GFAIC charter alignment (attestation, see Section 9.8.2)
4. Generate full compliance report for auditors
5. Archive report with timestamp and attestation

9.8.1 Applicable AIVerify plugin subset

The AIVerify stock plugin set (v2.x) ships 11 plugins. Not all are applicable to this deployment, which is a text-only enterprise finance / governance system (no images, no speech, no predictive classifiers on tabular data). The quarterly audit is scoped to the subset that is meaningful for the deployed agents; applying plugins outside this subset produces "N/A" results that dilute the audit trail without adding assurance.

Applicable subset: 7 of 11 (Robustness-text, Fairness-text, Explainability-LIME, Performance-accuracy, Hallucination, PII leakage, Toxicity). The other 4 plugins are documented as `not_applicable` with the rationale above; the quarterly compliance report records the applicability decision explicitly so that auditors can challenge the scoping if the deployment changes (e.g. if tabular classifiers are added in v2.0, the fairness and explainability tabular plugins become applicable).

Table 9.2: AIVerify plugin applicability for this deployment.

Plugin	Applicable	Rationale
Robustness – text adversarial	Yes	Deployed agents consume text inputs; adversarial tests exercise prompt-injection surface.
Robustness – image adversarial	No	No image inputs in scope.
Fairness – tabular classifier	No	No classifier with protected-attribute inputs; the TB commentary model does not classify.
Fairness – text	Yes	Applied to Arabic-language outputs to detect language-based quality disparity.
Explainability – SHAP tabular	No	Not applicable (no tabular classifier).
Explainability – LIME text	Yes	Applied to TB commentary: used to validate that cited variance drivers match commentary.
Performance – accuracy	Yes	Generic accuracy/benchmark against golden corpus.
Performance – confusion matrix	No	No multi-class classifier.
Hallucination / groundedness	Yes	Core test for RAG-backed commentary.
PII leakage	Yes	Critical for Arabic AP (invoices) and TB (entity data).
Toxicity / safety	Yes	Required by MAS MGF regardless of domain; low expected firing rate but must be part of the audit.

9.8.2 GFAIC charter attestation

The full GFAIC (Group Financial AI Committee) charter is maintained in the internal governance repository for confidentiality. For external auditors, this specification provides two verifiable deliverables:

Redacted summary. `docs/governance/gfaic-charter-redacted.md` — a public-facing summary with sensitive member names, routing escalations, and budget figures redacted. Covers: charter scope, standing members (by role, not name), meeting cadence, decision rights, escalation to the Board.

Signed attestation. `docs/governance/gfaic-attestation.pdf` — a one-page document signed quarterly by the GFAIC chairperson attesting that: (a) the full charter on the internal repository has not changed materially since the previous attestation, (b) the AI systems described in this specification are within GFAIC scope, (c) GFAIC reviewed the last quarterly compliance report. The attestation's SHA-256 is recorded in the registry and in this specification's build metadata.

External auditors requiring the full charter can request it via the Audit & Risk Committee under NDA; the attestation plus redacted summary is sufficient for routine regulatory review.

9.9 Acceptance criteria

Tests pass when:

1. All P1 test cases pass (100%)
2. P2 test cases pass ($\geq$95%)
3. Line coverage $\geq$90%
4. All schemas validate
5. Coverage $\geq$75% for all source documents

6. No “must” requirements in gap status
7. All implementation pointers resolve

Chapter 10

Per-Request Identity Attribution Specification

This chapter specifies the per-request identity attribution system, addressing the MAS MGF requirement REG-MGF-2.1.2-002 that "each agent action must be traceable to the requesting identity and approval chain."

10.1 Overview

Per-request identity attribution ensures that every AI agent action can be traced to:

1. The **initiating user** who triggered the request
2. The **agent identity** that executed the action
3. The **approval chain** (if human-in-the-loop was invoked)
4. The **system context** (pipeline, model version, timestamp)

10.2 Identity model

10.2.1 Agent identity schema

Listing 10.1: Agent identity record schema.

```
{
  "agent_id": "agent-tb-variance-001",
  "agent_type": "variance_analyzer",
  "model_version": "qwen2.5-72b-instruct-v1.2",
  "pipeline_version": "tb-review-2.1.0",
  "deployment_environment": "production",
  "capabilities": [
    "variance_calculation",
    "anomaly_detection",
    "commentary_generation"
```

```
  ],
  "permissions": {
    "can_modify_data": false,
    "can_trigger_downstream": true,
    "requires_human_approval": ["commentary_generation"]
  }
}
```

10.2.2 Request identity schema

Listing 10.2: Request identity record schema.

```
{
  "request_id": "req-20260418-001234",
  "correlation_id": "session-abc123",
  "initiating_user": {
    "user_id": "jsmith@company.com",
    "user_type": "human",
    "authentication_method": "oauth2_sso",
    "roles": ["tb_reviewer", "senior_accountant"]
  },
  "timestamp": "2026-04-18T10:30:00+08:00",
  "source_system": "tb-review-dashboard",
  "source_ip": "10.0.1.100"
}
```

10.3 Audit trail specification

10.3.1 Audit event schema

Every agent action generates an audit event:

Listing 10.3: Audit event schema.

```
{
  "event_id": "evt-20260418-567890",
  "event_type": "agent_action",
  "timestamp": "2026-04-18T10:30:05+08:00",

  "request_identity": {
    "request_id": "req-20260418-001234",
    "initiating_user_id": "jsmith@company.com"
  },

  "agent_identity": {
    "agent_id": "agent-tb-variance-001",
    "model_version": "qwen2.5-72b-instruct-v1.2"
```

```
  },

  "action": {
    "action_type": "variance_calculation",
    "entity_id": "HKG-2026-04",
    "input_hash": "sha256:abc123...",
    "output_hash": "sha256:def456..."
  },

  "approval_chain": [
    {
      "approver_id": "jsmith@company.com",
      "approval_type": "implicit",
      "timestamp": "2026-04-18T10:30:00+08:00"
    }
  ],

  "outcome": {
    "status": "success",
    "duration_ms": 1234,
    "tokens_used": 500
  }
}
```

10.3.2 Required audit events

Table 10.1: Mandatory audit events by action type.

Action Type	Event Type	Required Fields
AI inference	`agent_inference`	model_version, input_hash, output_hash
Data modification	`agent_write`	entity_id, before_hash, after_hash
Human review request	`review_requested`	reviewer_id, queue_id
Human approval	`review_approved`	approver_id, decision, rationale
Human rejection	`review_rejected`	rejecter_id, decision, rationale
Error/failure	`agent_error`	error_code, stack_trace_hash

10.4 Storage and retention

10.4.1 Storage requirements

Table 10.2: Audit storage requirements.

Data Type	Retention	Storage
Audit events	7 years	Append-only log (immutable)
Request identities	7 years	Indexed for query
Agent configurations	Forever	Version-controlled
Input/output hashes	7 years	With audit events
Full input/output	90 days	Cold storage, then purge

10.4.2 Immutability requirements

Audit logs **must** be immutable:

- Append-only storage (no updates, no deletes)
- Cryptographic chaining (hash of previous event included)
- External timestamp authority (prevents backdating)
- Write-once media for long-term archive

10.5 API specification

10.5.1 Audit write API

Two audit write points are required per action: a `pre_action` reservation (before the inference executes) and a `post_action` completion (after the inference returns). The `pre_action` write is the enforcement gate — if it fails, the inference is *rejected before execution*, which is the only tractable way to achieve the "no action without audit" invariant (see Section 10.6.2).

Listing 10.4: Audit service API contract.

```
def reserve_agent_action(
    request_identity: RequestIdentity,
    agent_identity: AgentIdentity,
    planned_action: AgentActionPlan,
) -> AuditEventId:
    """
    Reserve an audit event slot BEFORE the agent action executes.

    This function MUST be called and succeed before the inference is
    dispatched. A successful return contractually commits the caller
    to writing a matching complete_agent_action() within the event's
    TTL (default: 120 s).

    Raises:
        AuditWriteError: If the reservation cannot be written. The
            caller MUST NOT dispatch the agent action.
    """

def complete_agent_action(
    event_id: AuditEventId,
    action: AgentAction,
    outcome: ActionOutcome,
) -> None:
    """
    Complete a previously reserved audit event with the actual
    action and outcome. MUST be called after the action returns
    (whether success or error).

```

```
    If this call fails, the reservation remains visible in the audit
    trail in state "orphaned" and operations are alerted; the caller
    MUST return an error to the user because downstream side effects
    could not be verifiably attributed.
    """
```

10.5.2 Audit query API

Listing 10.5: Audit query API contract.

```
def query_audit_trail(
    filters: AuditFilters,
    pagination: Pagination
) -> AuditQueryResult:
    """
    Query the audit trail.

    Filters:
        - request_id: Specific request
        - user_id: All actions by user
        - agent_id: All actions by agent
        - entity_id: All actions on entity
        - time_range: Start and end timestamps
        - action_type: Filter by action type

    Returns:
        List of matching audit events with pagination
    """
```

10.6 Integration requirements

10.6.1 Pipeline integration

Every AI pipeline **must** integrate the identity attribution system:

1. **Request entry**: Capture initiating user identity from authentication
2. **Pre-inference**: Log `agent_inference_start` event
3. **Post-inference**: Log `agent_inference_complete` event
4. **Data write**: Log `agent_write` event before commit
5. **Error**: Log `agent_error` event on any failure

10.6.2 Failure handling

LLM inference is not reversible once executed: tokens have been emitted, compute was consumed, and any downstream side effects (database writes, external calls) are already in flight. "Roll back the inference" is therefore not a valid operation. The specification enforces

audit coverage by *gating* the action behind a successful reservation write, and by containing side effects:

1. **Reservation-before-action.** The `reserve_agent_action()` call (see audit write API) MUST succeed before the agent dispatches the inference. If the reservation write fails, the agent action MUST NOT be executed and the request MUST return a `503` error with an operations alert.
2. **Side-effect containment.** Any data-modification or external call triggered by an AI output MUST be staged and only committed after `complete_agent_action()` succeeds. If completion fails, staged side effects are discarded (this *is* a rollback — of the side effects, not of the inference itself).
3. **Orphaned reservation handling.** If a reservation is written but never completed within the TTL (e.g. the inference server crashed), the reservation is marked `orphaned`; operations are alerted; staged side effects are discarded; downstream consumers see an explicit failure.
4. **Alerting and no-silent-proceed.** An alert MUST be sent to operations on every audit failure; the system MUST NOT proceed to emit results without a completed audit record.

This wording supersedes the earlier text (which called for "rolling back the agent action"): because LLM inference is not reversible, the equivalent invariant is enforced by gating with a pre-action reservation and staging side effects.

10.7 Compliance mapping

Table 10.3: MAS MGF requirement mapping.

Requirement	Section	Implementation
REG-MGF-2.1.2-002	§10.2–10.3	Request identity schema, audit trail
REG-MGF-2.1.3-001	§10.4	Immutable storage, retention policy
REG-MGF-2.2.1-001	§10.5	Audit write API with rollback

10.8 Implementation checklist and integration status

10.8.1 Audit service (this chapter)

- [x] Agent identity schema defined
- [x] Request identity schema defined
- [x] Audit event schema defined
- [x] Audit write API contract specified (reserve/complete pattern, Section 10.6.2)
- [x] Audit query API contract specified
- [ ] Audit service implementation (`src/governance/audit_service/`) — target: v1.0

- [ ] Immutable storage configured (append-only, hash chaining, external timestamp authority) — target: v1.0
- [ ] Retention policy enforced (7 years) — target: v1.0

10.8.2 Pipeline integration rollout

The integration of the identity attribution audit service with the three application pipelines (TB, Arabic AP, Simula) is sequenced against explicit milestones, not an open-ended forward commitment. Each pipeline has its own integration sub-ticket with an acceptance criterion and a position in the dependency graph of Chapter 11; slippage on any row escalates to the GFAIC governance committee. **Sequence, not dates**: the rows below use the MoSCoW identifiers from Table 11.2 so implementation teams know what depends on what, not when a calendar target is supposed to fire (that belongs in the steering packet).

Table 10.4: Audit service and pipeline integration rollout, keyed to the MoSCoW priorities in Table 11.2. Pri. column: M=Must (critical path), S=Should, C=Could.

ID	Prereq	Pri.	Ticket	Acceptance criterion
R1	—	M	REG-AUDIT-001	Audit service skeleton (reserve/complete endpoints, in-memory store): contract tests pass against the two endpoints; 404/409/500 paths exercised.
R2	R1	M	REG-AUDIT-002	Immutable storage backend (append-only + hash chain + RFC 3161 timestamp): tamper-test suite passes; restored chain validates after adversarial edit.
R3	R2	M	REG-AUDIT-003	7-year retention + legal-hold API: retention policy enforced; legal-hold flag blocks GDPR erasure.
R4	R3	M	REG-INTG-TB-001	TB dashboard integration: every dashboard decision *and* AI commentary draft emits `reserve/complete`; 100 % coverage in Chapter 9.
R5	R4	S	REG-INTG-AP-001	Arabic AP integration: OCR, field extraction, VAT rule evaluation each emit audit events; PII hashed.
R6	R5	S	REG-INTG-SIM-001	Simula batch integration: training-data runs logged at batch granularity; per-example logging enabled under `AUDIT_VERBOSE=1`.
R7	R4	M	GAIC-REV	v1.0 compliance gate: quarterly GFAIC review confirms MAS MGF 2.1.2-002 compliance; traceability matrix Table 7.3 flips from `partial` to `implemented` on this row.

Until each pipeline's integration is verified, the governance status for that pipeline's MAS MGF 2.1.2-002 compliance is `partial` (see the traceability matrix in Chapter 7). Integration tickets REG-INTG-TB-001 (R4) is a Must for the Regulations project and a Must for TB; REG-INTG-AP-001 (R5) is a Should for Regulations and a Must for Arabic AP (it is the `partial`-to-`compliant` upgrade path for REG-MGF-2.1.2-002); REG-INTG-SIM-001 (R6) is Should for Regulations and is not a v1.0 gate for Simula itself. Table 10.4 is tracked in the programme status dashboard and refreshed on every cycle.

10.8.3 Compliance tests

- [ ] Per-pipeline compliance tests passing (Chapter 9, Section on audit coverage)
- [ ] Quarterly re-validation against the AIVerify subset (Section 9.8.1)

Chapter 11

Project Roadmap

Scope of this chapter

This chapter is the roadmap for the **Agentic AI Regulations corpus and implementation-evidence** project. It is a project-level artefact, not a programme plan. Other SAP OSS Finance projects (Trial Balance, Arabic AP, Simula) run on the same platform, workspaces, and backends, but each owns its own roadmap, business case, and go/no-go decision. This project is a *supplier* of shared-platform components (audit service, identity attribution, monitoring, conformance tooling) that the consumer projects depend on.

Sequence, not dates. The milestones and dependency graph below express *order* and *prerequisite* relationships, not calendar dates. This is deliberate. Implementation teams must know what depends on what, and where the schedule can absorb slippage (the Should/Could items); a date column would harden targets into commitments they were never meant to be. Dates, when set, live in the project steering packet and the ticketing system, not in this specification.

11.1 MoSCoW classification

Every milestone below carries one of four priorities. The mapping is contractual: a milestone's priority determines what happens when it is dropped, de-scoped, or slips.

The critical path of this project is the chain of Must items in Table 11.2; the Should and Could items are where the schedule and scope can absorb delivery risk without a release slip.

Table 11.1: MoSCoW prioritisation (shared across the four SAP OSS Finance projects).

Priority	Meaning
Must (**M**)	Non-negotiable for v1.0. Without this milestone, the release does not ship.
Should (**S**)	Important but not vital. Ship without it only with sponsor sign-off; document the deviation in the business-case r
Could (**C**)	Desirable enhancement. Included if Must and Should items are on track.
Won't (**W**)	Explicitly out of v1.0 scope. Listed so the boundary is legible.

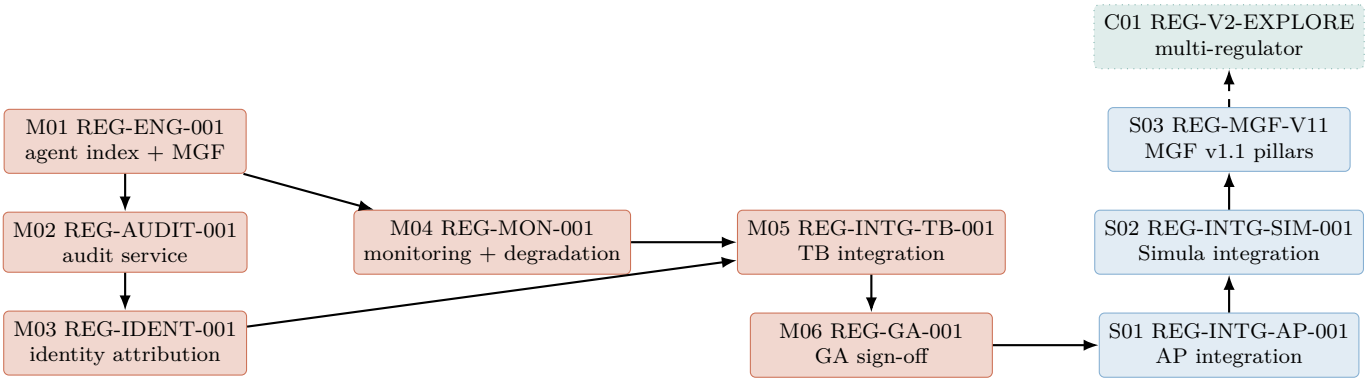

Figure 11.1: Regulations dependency graph. Amber nodes are Must (critical path); blue are Should (consumer-integration and MGF-pillar allowance); dotted green are Could (discretionary). Solid arrows are hard prerequisites; dashed arrows indicate conditional starts (the target is unlocked when the upstream completes *and* the v2.0 trigger in Section 11.7 fires).

Table 11.2: Regulations v1.0 project milestones in dependency order. Priority: M=Must (critical path), S=Should, C=Could, W=Won't.

ID	Prereq	Pri.	Ticket	Milestone
M01	—	M	REG-ENG-001	Agent index and MGF framework specification complete; conformance tooling passes the fixtures in **??**.
M02	M01	M	REG-AUDIT-001	Audit service (reserve/complete, immutable storage) live in UAT; SEV1/SEV2 clean.
M03	M02	M	REG-IDENT-001	Identity attribution wiring (Chapter 10) live in UAT; REG-MGF-2.1.2-002 compliance upgraded from *missing* to *partial*.
M04	M01	M	REG-MON-001	Monitoring spec operational: circuit breaker plus graceful-degradation ladder (Section 8.5.4).
M05	M03, M04	M	REG-INTG-TB-001	First consumer (Trial Balance) integrated end-to-end; reserve/complete coverage = 100 % on TB actions.
M06	M05	M	REG-GA-001	GA sign-off (audit + identity + monitoring) on the TB integration.
M07	M06	M	REG-BC-REFRESH	Business-case refresh and compliance attestation (first cycle after GA; then annual).
S01	M06	S	REG-INTG-AP-001	Arabic AP integration: reserve/complete on AP actions; REG-MGF-2.1.2-002 becomes *compliant* once AP live.
S02	S01	S	REG-INTG-SIM-001	Simula integration: reserve/complete on Simula corpus-generation actions.
S03	M06	S	REG-MGF-V11	MGF v1.1 pillar updates ingested and mapped to agent index.
C01	S03 + Section 11.7 trigger	C	REG-V2-EXPLORE	v2.0 multi-regulator correlation scoped (beyond MAS MGF).
C02	M06	C	REG-TUNE	Quarterly conformance-tooling retraining from accept/edit telemetry.
W01	—	W	REG-V1-REALTIME	Real-time cross-project audit dashboards are *not* in v1.0 scope.
W02	—	W	REG-V1-MULTIREG	Multi-regulator correlation is *not* in v1.0 scope; single-regulator (MAS MGF) only.

11.2 Dependency graph

11.3 Project milestones

The Must chain (M01 → M02 → M03/M04 → M05 → M06 → M07) is the critical path. The Should items (S01–S03) are the primary *schedule allowance*: additional consumer integrations and MGF pillars can slip without affecting v1.0 sign-off. The Could items are the *scope allowance*: they may be cut to v1.1 without renegotiation.

11.4 Launch tiers and exit gates

The project walks the four-tier ladder shared across SAP OSS Finance projects. A tier is exited only when the named exit gate is satisfied; slippage requires sponsor sign-off.

11.5 Kill-switch conditions

The kill-switch is a pre-committed rulebook: a tripped condition enters the action column without further debate. GFAIC and sponsor are notified within 24 hours. The conditions are evaluated continuously (on every cycle) and do not carry dates.

11.6 Shared-platform dependencies

This project depends on two shared-platform components (it supplies the rest). Each is tracked here only as a dependency, not a commitment this project can unilaterally adjust. Sequence is expressed in terms of the Must item that first consumes the dependency, not in calendar time.

Shared-platform dependencies are surfaced at monthly steering. If any upstream owner declares **amber** or **red** on a component in Table 11.5, the affected downstream Must milestone is re-planned at the following steering.

11.7 v2.0 expansion trigger

The Could item C01 (REG-V2-EXPLORE) is unlocked only when all of the following are observed after two consecutive quarters of full rollout under v1.0:

- a second regulator framework (beyond MAS MGF) is formally adopted as in-scope by the AI Governance Working Group;
- MGF v1.1 pillars (S03) are fully mapped and at *compliant* status;
- business-case refresh (M07) explicitly budgets the v2.0 programme.

If any of the three conditions fails, C01 remains in the Could column and is re-evaluated at the next business-case refresh.

Table 11.3: Regulations launch tiers. The exit gate defines the MoSCoW items that must be closed to exit the tier.

Tier	**Scope**	**Exit gate (references MoSCoW IDs)**
Alpha	Engineering only, DEV + UAT, conformance fixtures only.	M01, M02, M04 closed; conformance tooling green on fixtures; reproducibility of audit outputs confirmed on two consecutive runs.
Beta	Audit + identity + monitoring live for one consumer project (TB) in UAT.	M03, M05 closed; parallel-run gate in the TB spec passes for one full cycle; no open SEV1/SEV2; audit coverage = 100 % of in-scope TB actions.
GA (limited)	Audit + identity + monitoring live for TB in production.	M06 closed; Beta gate sustained for two consecutive cycles; kill-switch green; REG-MGF-2.1.2-002 at *partial* (TB-only).
Full rollout	All three current consumer projects integrated (TB, AP, Simula).	S01 and S02 closed; REG-MGF-2.1.2-002 at *compliant*; two quarterly reviews sustained; M07 complete; business-case refresh complete.

Table 11.4: Regulations kill-switch.

Condition	Evaluation	Action
Audit-coverage gap (missed `reserve/complete` pair) detected in production	continuous	**Halt GA onboarding for all consumers**; incident-review before next integration.
Circuit breaker tripped with no successful graceful-degradation fallback for three consecutive cycles	per cycle	**Halt upstream LLM traffic**; fail over to cached/heuristic path; remediate before resuming.
Identity-attribution reconciliation fails (principal of record cannot be resolved) for $> 0.01\,\%$ of actions	per cycle	**Pause affected consumer**; block further reserves until remediated.
Any MAS MGF finding attributable to the Regulations platform (audit, identity, monitoring)	continuous	**Halt rollout**; remediate before next wave.

Table 11.5: Shared-platform dependencies.

Component	Owner	First consumed by	Impact if late
Unified schema registry (`docs/schema/`)	Shared infra	M01	Already `migration.status=complete`; no remaining risk.
vLLM hosting (`vllm-only` policy)	Shared platform	M04	The graceful-degradation ladder (Section 8.5.4) is the mitigation; this project is the *owner* of that ladder for sibling consumers.

11.8 Governance and business-case cadence

Project governance. The Regulations project reports monthly to a project steering (AI Governance Lead, Compliance Officer, Platform Engineer) and quarterly to the cross-project GFAIC, which it also convenes. Monthly steering accepts or rejects amber/red signals on Must items; GFAIC reviews compliance posture and cross-project dependencies (this project's consumer-integration Should items feed directly into sibling projects' Must items).

Business-case refresh (M07). The business case that authorises this project is refreshed on the following rhythm:

- **Mandatory refresh**: at the first cycle after GA, and annually thereafter (ticket REG-BC-REFRESH, priority M in Table 11.2). Refresh packet includes consumer integration load assumptions, unit economics (**??**), platform operating cost realised vs. forecast, and a deviation statement against the baseline.
- **Trigger-based refresh**: any of
  - consumer-project count in scope changes by ≥ 1 (new consumer adopts, or one is retired);
  - per-action audit cost changes by $\geq 30\,\%$ (storage, indexing, identity-reconciliation throughput);
  - a new MGF pillar, a new regulator framework, or a material change to REG-MGF-2.1.2-002;
  - any kill-switch in Table 11.4 is tripped.
- **Owner**: AI Governance Lead. Approval routed through the project steering and filed at `docs/regulations/business-case/refresh-history.yaml`.